Object Oriented Programming Concepts

Classes and Objects • Polymorphism • Encapsulation
Inheritance • Types of Methods • Hands-on Program

Concepts, examples and guided practice

Object-Oriented Programming (OOP)

THE IDEA

OOP organizes software around objects that combine data (attributes) and behaviour (methods) in one cohesive unit.

Building blocks

C

CLASS = blueprint

Defines attributes and methods.

O

OBJECT = instance

A specific entity created from a class.

Example: Car class → Red Tesla object

The four pillars of OOP

1

ENCAPSULATION

Bundles data + methods; controls direct access.

2

ABSTRACTION

Hides complexity; exposes essential features.

3

INHERITANCE

Derives features from an existing class.

4

POLYMORPHISM

Same method/interface, different behaviour.

Why use OOP?

REUSE

Use existing code

MAINTENANCE

Simpler to update

PROTECTION

Encapsulation safeguards data

IN A NUTSHELL

A class is the blueprint; objects are its instances. OOP supports reusable, modular and maintainable software.

Classes and Objects

Class

A blueprint that defines the **attributes** and **methods** its objects can have.

Object

An instance of a class. Multiple objects can hold different values.

Example: define a class and create objects

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print(self.name, self.marks)
s1 = Student("Ananya", 85)
s2 = Student("Rahul", 92)
s1.display()
s2.display()
```

Key points

__init__() initializes an object.

self refers to the current object.

self.name and self.marks are instance attributes.

Dot notation accesses attributes and methods.

Class and Instance Attributes

The key difference is whether a value is shared by the class or belongs to one object.

CLASS ATTRIBUTE

Student.college_name → shared value

INSTANCE ATTRIBUTE

s1.name / s2.name → object-specific

Python example

```
class Student:
    college_name = "Bangalore College"

    def __init__(self, name):
        self.name = name

s1 = Student("Kiran")
s2 = Student("Mahesh")

print(s1.name, s2.name)
print(Student.college_name)
print(s1.college_name)
```

OUTPUT

```
karan arjun
ABC College
ABC College
```

If an instance attribute has the same name as a class attribute, the instance value is used for that object.

Practice 1: Student Average

Task: Store a student's name and three marks, then calculate and print the average using a method.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def get_avg(self):
        avg = sum(self.marks) / len(self.marks)
        print(self.name, "average:", avg)

s1 = Student("John Mathew", [99, 98, 97])
s1.get_avg()
```

OUTPUT

```
John Mathew average: 98.0
```

What this reinforces

- **Constructor**

  stores the name and marks

- **List operations**

  sum() and len() calculate the average

- **Instance method**

  uses self to access the student's data

Inheritance in Python

What is inheritance?

Inheritance allows a new class to acquire properties and methods from an existing class.

Important terminology

Parent class — Base class / Superclass

Child class — Derived class / Subclass

Why use inheritance?

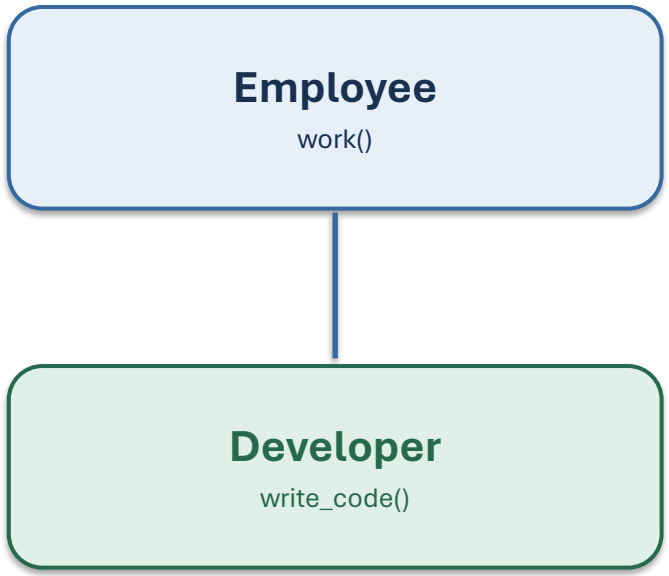
- Reuse common code instead of duplicating it.
- Extend an existing class with new behaviour.
- Override inherited methods when required.

Basic Python syntax

```
class Child(Parent):
    statements
```

Single Inheritance

One child class inherits from one parent class.



Developer can use its own method and the inherited work()

Python example

```
class Employee:
    def work(self):
        print("Performs work")

class Developer(Employee):
    def write_code(self):
        print("Writes code")

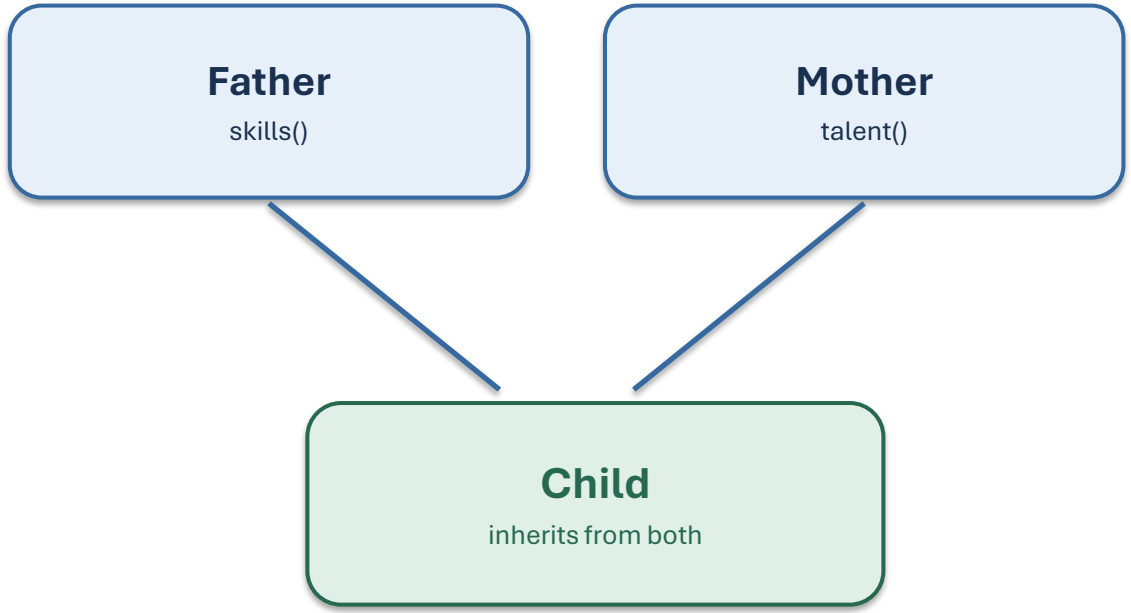
d = Developer()
d.work()
d.write_code()
```

Output

```
Performs work
Writes code
```

Multiple Inheritance

One child class inherits from two or more parent classes.



The child can access methods from both parent classes.

Python example

```
class Father:
    def skills(self):
        print("Driving")

class Mother:
    def talent(self):
        print("Cooking")

class Child(Father, Mother):
    pass

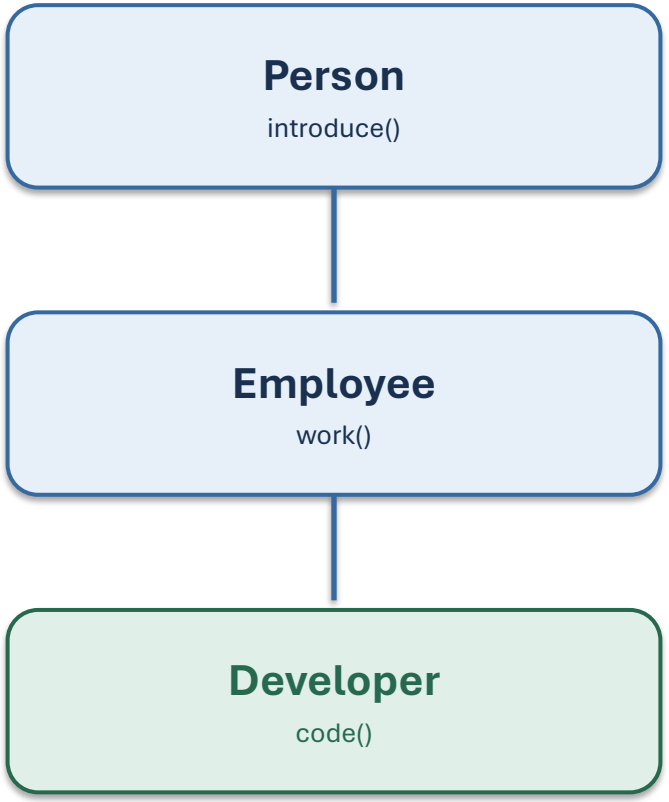
c = Child()
c.skills()
c.talent()
```

Output

```
Driving
Cooking
```

Multilevel Inheritance

A class inherits from another derived class, forming an inheritance chain.



Developer inherits from Employee and Person.

Python example

```
class Person:
    def introduce(self):
        print("I am a person")

class Employee(Person):
    def work(self):
        print("I work")

class Developer(Employee):
    def code(self):
        print("I write code")

d = Developer()
d.introduce()
d.work()
d.code()
```

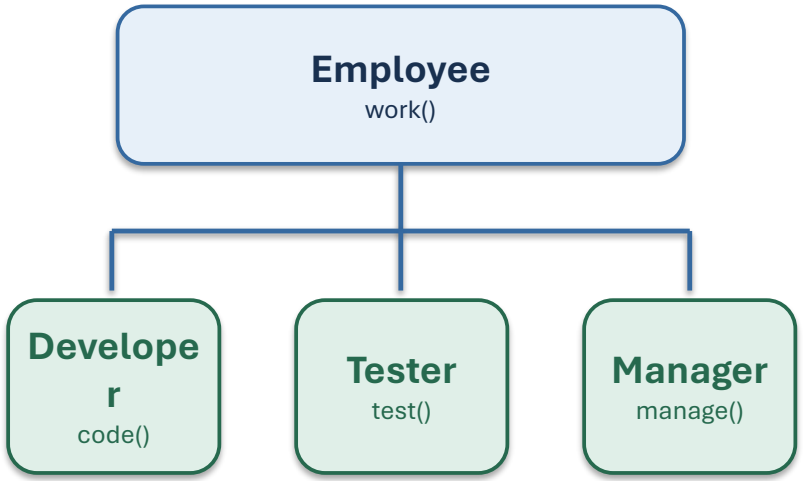
Output

```
I am a person
I work
I write code
```

Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Inheritance diagram



Each child class inherits work() from Employee and can also define its own methods.

Python example

```
class Employee:
    def work(self):
        print("Performs work")

class Developer(Employee):
    pass

class Tester(Employee):
    pass

class Manager(Employee):
    pass

d = Developer()
t = Tester()
m = Manager()

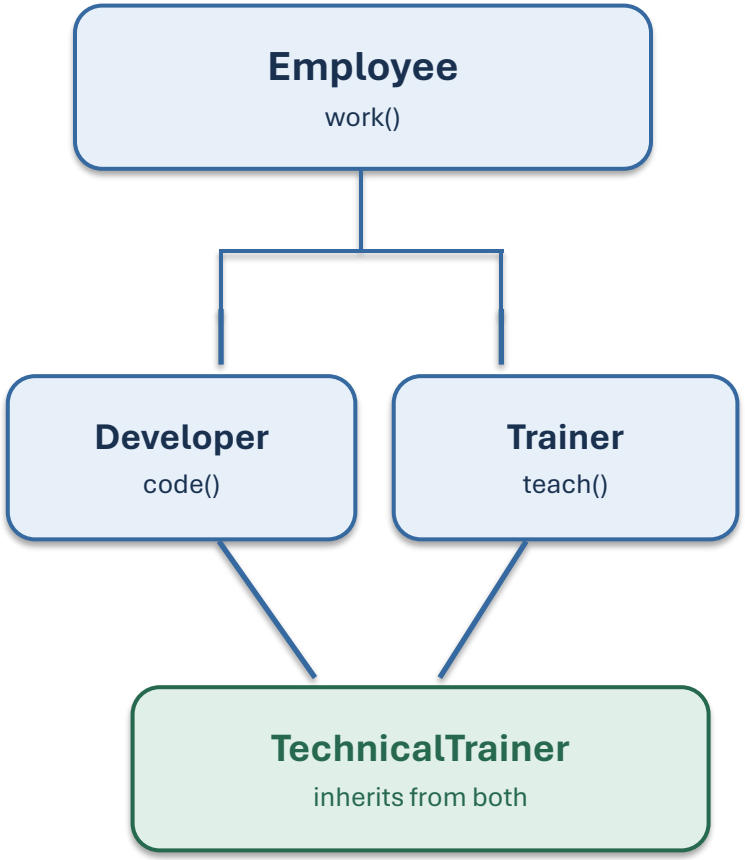
d.work()
t.work()
m.work()
```

Output

```
Performs work
Performs work
Performs work
```

Hybrid Inheritance

A combination of two or more inheritance types.



Hierarchical + Multiple inheritance

Python resolves method lookup using MRO.

Python example

```
class Employee:
    def work(self):
        print("Performs work")

class Developer(Employee):
    def code(self):
        print("Writes code")

class Trainer(Employee):
    def teach(self):
        print("Teaches students")

class TechnicalTrainer(Developer, Trainer):
    pass

t = TechnicalTrainer()
t.work()
t.code()
t.teach()
```

Output

```
Performs work
Writes code
Teaches students
```

Polymorphism in Python

Definition

Polymorphism is an Object-Oriented Programming concept that allows different objects to respond to the same method call or operation in their own specific way.

Meaning of the word

Poly means “many” and morph means “forms”. Polymorphism literally means “many forms”.

Simple illustration – same method, different behaviour

Dog → sound() → Barks
Cat → sound() → Meows
Cow → sound() → Moos

Key takeaway

Same method name, different behaviour depending on the object.

Method Overriding

Concept

When a child class provides its own implementation of a method already defined in the parent class, it is called method overriding.

Python example

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

dog = Dog()
dog.sound()
```

Expected output

```
Dog barks
```

Key takeaway

The child class changes the behaviour of an inherited method by providing its own implementation.

Duck Typing

Concept

Python focuses on what an object can do rather than its actual class or type. Objects that support the required operation can be used in the same way.

It comes from the saying: “If it walks like a duck and quacks like a duck, it must be a duck.” The idea is that we identify something by what it does, rather than by checking what it is.

Python example

```
class CashPayment:
    def pay(self):
        print("Cash payment")

class UpiPayment:
    def pay(self):
        print("UPI payment")

def process(payment):
    payment.pay()

process(CashPayment())
process(UpiPayment())
```

Expected output

```
Cash payment
UPI payment
```

Key takeaway

Python looks for the required behaviour (such as a pay() method), not a specific class or inheritance relationship.

Built-in Function Polymorphism

Concept

The same built-in function can work with different types of objects and produce a result appropriate to each object.

Python example

```
print(len("Python"))
print(len([10, 20, 30]))
print(len({"A": 1, "B": 2}))
```

Expected output

```
6
3
2
```

Key takeaway

len() works with a string, list and dictionary, returning the length appropriate to each type.

Operator Overloading

Concept

The same operator can perform different operations depending on the types of its operands.

Python example

```
print(10 + 20)
print("Hello " + "Python")
print([10, 20] + [30, 40])
```

Expected output

```
30
Hello Python
[10, 20, 30, 40]
```

Key takeaway

+ performs addition with numbers, concatenation with strings, and combines lists.

Abstract Interfaces

Concept

An abstract class defines a common structure. Child classes must implement its abstract methods before their objects can be created.

Python example

```
from abc import ABC, abstractmethod # ABC – Abstract Base Class

class Payment(ABC):
    @abstractmethod
    def pay(self):
        pass

class CashPayment(Payment):
    def pay(self):
        print("Paid using Cash")

class UpiPayment(Payment):
    def pay(self):
        print("Paid using UPI")

CashPayment().pay()
UpiPayment().pay()
```

Expected output

```
Paid using Cash
Paid using UPI
```

Key takeaway

The abstract class specifies what must be implemented; each child class decides how to implement it.

Abstraction in Python: Abstract Classes & Methods

1 THE CORE IDEA

Abstraction defines WHAT a class must do — not HOW it does it.

An abstract parent class sets a common requirement for its child classes.

2 ABSTRACT CLASS & METHOD

```
class Payment(ABC):
    @abstractmethod
    def pay(self):
        pass
```

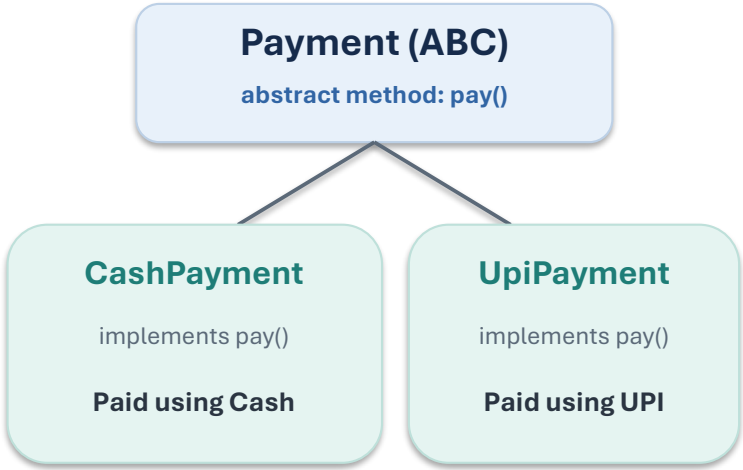
ABC: base for an abstract class

@abstractmethod: marks pay() as required

pass: placeholder; no payment logic here

The abstract parent describes the required operation; it does not perform the payment.

3 CHILD CLASSES IMPLEMENT pay()



4 OBJECT CREATION & METHOD CALLS

```
CashPayment().pay()
UpiPayment().pay()
```

Output

Paid using Cash | Paid using UPI

5 WHY IMPLEMENTATION MATTERS

A child class must implement every inherited abstract method before it can be instantiated.

```
class CardPayment(Payment):
    pass

CardPayment()  # TypeError
```

Because pay() is still unimplemented, Python prevents creation of the object.

6 KEY DISTINCTION

Abstract class: declares the contract.
Child class: supplies the implementation.

REMEMBER

Different payment classes implement the same required pay() operation in their own way.

What is Encapsulation?

DEFINITION

Combining data and methods within a single class while controlling access to internal data.

Why it matters

- **Invariants stay intact.** The class owns its rules (a balance can't go negative, a status can't be set to an invalid value), so every caller gets the same guarantees.
- **Implementation can change freely.** Callers depend on the public interface, not the internals, so you can refactor storage or logic without breaking consumers.
- **Smaller surface area.** Fewer things are exposed, so there are fewer ways to misuse the object and less to maintain.

Access Levels in Python

Convention	Meaning	Enforced?
`name`	Public, part of the interface	n/a
`_name`	Protected/internal, "don't touch unless you're a subclass or the owner"	Convention only
`__name`	Name-mangled to `_ClassName__name` to avoid accidental clashes in subclasses	Mangled, but still reachable

Double underscore is about avoiding subclass collisions, not security. Anyone can still reach it via `obj._ClassName__name`.

Access Levels in Python

Python uses naming conventions to indicate how class attributes should be accessed.

```
class Employee:
    def __init__(self):
        self.name = "John"          # Public
        self._department = "IT"     # Protected
        self.__salary = 50000       # Private

    def display_details(self):
        print(self.name)
        print(self._department)
        print(self.__salary)


emp = Employee()

print(emp.name)
print(emp._department)
# print(emp.__salary)  # AttributeError

emp.display_details()
```

PUBLIC

name

Accessible from anywhere.

PROTECTED

_department

Intended for internal use and subclasses.

PRIVATE

__salary

Name mangling discourages direct access.

TRY THIS!

```
emp._Employee__salary
```

Output: 50000 | Name mangling

Key Takeaways

- Combines data and methods within a class
- Controls access to internal data
- Uses public, protected and private naming conventions
- Enables validation through methods
- Improves maintainability and reliability

REMEMBER

Encapsulation = Data + Methods + Controlled Access

Types of Methods in Python

Instance method

A method that operates on the data of a specific object

- First parameter: self
- Works with an object's data
- Example: display_details()

Use it for object-specific behavior.

Class method

A method that operates on class-level data shared by all objects

- First parameter: cls
- Uses @classmethod
- Works with class-level data

Use it for shared class information or alternative constructors.

Static method

A method that performs a related operation independently but does not receive an automatic reference to the instance or class.

- No automatic self or cls
- Uses @staticmethod
- Example: is_valid_salary()

Use it for a related utility operation.

Memory aid

Instance = object data | Class = class data | Static = utility behavior

Program Example - Practical Implementation of Methods Types using an Employee Class

```
class Employee:

    # Class variable
    company = "ABC Technologies"
    employee_count = 0
    min_sal = 10000

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.employee_count += 1

    # Instance method
    def display_details(self):
        print("Employee:", self.name)
        print("Salary:", self.salary)
        print("Company:", Employee.company)

     # Class method
    @classmethod
    def change_company(cls, new_company):
        cls.company = new_company
        print("\nCompany name updated successfully!")

     # Static method
    @staticmethod
    def is_valid_salary(salary):
        return salary >= Employee.min_sal
```

Program – Contd

```
# Create employee objects
emp1 = Employee("John", 50000)
emp2 = Employee("Mary", 40000)

print ("name:", emp1.name, " Salary :", emp1.salary)
print("John's salary valid:", Employee.is_valid_salary(emp1.salary))

# 1. Instance method
print("--- Employee Details ---")
emp1.display_details()
emp2.display_details()

# 2. Class method
Employee.change_company("XYZ Solutions")

print("\n--- After Changing Company ---")
emp1.display_details()
emp2.display_details()

# 3. Static method
print("\n--- Salary Validation ---")
print("John's salary valid:", Employee.is_valid_salary(emp1.salary))
print("Mary's salary valid:", Employee.is_valid_salary(emp2.salary))

# Display total employee count
print("\nTotal Employees:", Employee.employee_count)
```

Student Practice

Practice exercises

1. Create a Student class with name and marks.
2. Add deposit and withdraw methods to BankAccount class.
3. Create Vehicle, Car and Bike classes using inheritance.
4. Implement sound() in Dog and Cat, then call it in a loop.
5. Add a class method and a static method to Calculator class.